MonthlyMuse

Project Progress Report — full MVP delivered, deployed and model-backed

AI-Powered Automated Message Generator for Monthly Posting · status as of November 2026 · companion to the Master Blueprint

MVP complete | 166 tests green | deployed — Vercel + Render | hosted LLM in production | template fallback, opt-in only | CI on every push

1. Executive summary

The project is **feature-complete against the blueprint's MVP definition (Section 15.1)**, and it is **deployed and serving live traffic**: the frontend on Vercel, the API and scheduler worker on Render, with a hosted language model configured in production so candidates are written by an LLM rather than by templates. A user can register, define a Monthly Plan, receive three ranked message candidates with transparent score breakdowns and full provenance, select/edit/reject one, watch the personalization engine update, and see the effect proven in analytics. The scheduler prepares each month's cycle automatically and sends T-3/T-1 reminders.

> **Verification status:** 166/166 automated tests pass (including every golden value from Section 17), TypeScript typecheck and production build are clean, CI is green on Python 3.10 and 3.11 plus the frontend build, and the deployed application was exercised end to end — login → dashboard → review → feedback — against the live API. Production health reports `llm_provider: groq`, `llm_model: openai/gpt-oss-120b`, `environment: production`, and the git commit it is running.

166 automated tests passing	**58** route handlers (45 paths)	**16** database tables	**12** frontend routes
14 AI pipeline stages	**137** files in repository		
7.9k lines of Python	**3.9k** lines of TypeScript/TSX	**6** LLM providers supported	**64** fallback templates
800 tone training rows	**100%** MVP scope delivered		

2. Blueprint coverage

Status of every major section of the Master Blueprint.

Blueprint section	Status	What exists
§3 Product & UX (screens)	**Complete**	10 product screens + root + review resolver: login, register, dashboard, create, review/compare, calendar, plans, history, analytics, settings
§4 AI/NLP pipeline (14 stages)	**Complete**	All fourteen numbered stages in `app/ai/pipeline.py`: validate → context → preprocess → intent vector → retrieval → arms → prompt → generate → filter → embed → features → score → MMR → confidence/explain
§5 Mathematical core	**Complete**	`s = F·w`, constrained gradient step, MMR, Dirichlet Monte-Carlo, Wilson interval, Thompson sampling, softmax tone classifier, chi-square, entropy, PCA
§8 Scheduler worker	**Complete**	APScheduler: daily cycle preparation (idempotent), T-3/T-1 reminders, catch-up, plan extension, `job_runs` audit, plus an authenticated webhook tick for hosts that suspend background schedulers
§9 Database schema	**Complete**	All 16 tables, portable string enums, UTC-normalised timestamps, float32 vector BLOBs
§10 API design	**Complete**	58 route handlers over 45 paths, uniform error envelope, JWT access + rotating refresh cookie, rate limits, request-id logging
§11 Frontend	**Complete**	Next.js 14 App Router, TanStack Query polling, RHF+Zod forms, Recharts visualisations
§12 Design system	**Complete**	Indigo token scale, component classes, reduced-motion support, responsive layout + mobile tab bar
§13 Architecture	**Complete**	Stateless API, separate worker, one database — deployed as frontend + API + worker
§14 Technology stack	**Complete**	As specified, with documented deviations (see §7) — notably a hosted LLM in the hot path
§15.1 MVP	**Complete**	Every row of the MVP table implemented and covered by tests
§16 Evaluation & analytics	**Complete**	Usage / Content / Quality dashboards: Wilson CIs, chi-square, novelty, entropy, MRR, top-1 agreement, correlation
§17 Golden tests	**Complete**	35 golden-value assertions, all passing
§19 Roadmap	**Complete**	All planned build stages delivered, plus deployment
Appendix B (repo, CI, Docker)	**Complete**	README, docker-compose, Dockerfiles, GitHub Actions CI, 4 notebooks
§15.2 Version 2	**Not started**	Platform publisher, engagement import, teams, dark mode, autopilot — intentionally out of MVP scope

3. Feature-by-feature completion

Feature	Done	Tests	Detail
Accounts & auth	**100%**	API journey	Register, login, logout, refresh rotation, Argon2id hashing, profile, timezone, data export, account delete
Monthly Plans	**100%**	API + scheduler	12-cycle creation, pause/resume/delete, auto-extension, next-generation computation; explicit start dates independent of the wall clock
Create Post form	**100%**	UI verified	3-step form, draft autosave, tone chips with Bayesian hints, occasion/keyword chips, "what the AI will use" panel
Generation pipeline	**100%**	13 pipeline + 29 behaviour	One LLM call over-generates 6 candidates → hard-constraint filter → dedup → embeddings → features → score → MMR → top 3; 202 + polling pattern
LLM adapters	**100%**	Unit + integration	Groq (production), Gemini, OpenAI, Anthropic, Ollama, template; per-provider circuit breakers, retry 1/3/9 s + jitter, tolerant JSON parse, cost metering, Groq model fallback list
Generation provenance	**100%**	Regeneration suite	Every request stores provider, model, source (schedule/regenerate/manual), created_at and fallback reason; the review header renders them
Brief persistence	**100%**	Regeneration suite	The brief a batch was built from is stored and read back — never reconstructed from candidate text; missing fields are hidden, not invented
Regeneration	**100%**	Regeneration suite	Regenerate-all re-runs the pipeline from the saved brief with single-flight protection; failures keep the previous batch and record the reason
Occasion gating	**100%**	22 occasion tests	Relevance (name-token coverage ≥ 0.5, narrow festive-intent bridge) + deterministic date eligibility; unrelated objectives never get a holiday bolted on
Review & compare	**100%**	Browser-verified	3 ranked cards, score rings, confidence chips, similarity meters, why-ranked text, expandable feature bars, radar compare, PCA embedding map, filtered-candidate disclosure
Candidate actions	**100%**	API + browser	Select, edit, copy, like, dislike, reject-with-reason, regenerate — each writes a feedback event
Personalization	**100%**	Math golden tests	Preference vector (EMA), Beta arms, learned length stats, online weight updates, reason nudges, learned-values UI with reset
Calendar	**100%**	Scheduler tests	Month grid with status chips, cycle detail modal, generate-now/skip/reschedule, signed .ics subscription
History & search	**100%**	Service tests	Text search + semantic (embedding) search, filters, detail drawer with batch context, reuse
Analytics	**100%**	Service tests	Usage, Content, Quality tabs; Wilson CI bars, chi-square panel, novelty trend, entropy, MRR, correlation
Web context (Exa)	**100%**	Optional integration	Occasion research through Exa when `EXA_API_KEY` is set; degrades silently to no web context, never blocks generation

Notifications	**100%**	Scheduler tests	In-app notifications + email (console backend in dev), reminder schedule
Settings	**100%**	UI verified	Voice, banned words, emoji, length, lead time, variety slider, pause learning, audience CRUD, export, delete
Operations	**100%**	Health tests	`/admin/health` reports provider, model, embedder, database, environment and deployed commit; `/admin/jobs` exposes the scheduler audit

4. Quality assurance

Suite	Count	Coverage
`test_math.py`	35	Every golden value in Section 17: feature/score vector, gradient steps (w_N→0.161, w_P→0.195, w_L→0.094), MMR order [A,C,F], Wilson CI (3,10)=(0.108,0.603), choice probabilities ≈[0.46,0.34,0.20]
`test_pipeline_behavior.py`	29	Provider-aware generation, no silent template substitution, fallback opt-in, provenance fields, filtered-candidate disclosure, confidence seeding
`test_occasion_gates.py`	22	Relevance threshold, festive-intent bridge, unrelated objectives, past-date eligibility, explicit retrospective choice
`test_plan_timing.py`	19	Cycle dates, timezone handling, explicit start dates independent of the real clock, horizon filling
`test_pipeline.py`	13	Hard constraints, length limits, banned words, dedup, arm-aware MMR diversity, stage payload shape
`test_text_quality.py`	12	Template-free copy checks: no placeholder leakage, banned words, CTA presence, length bounds
`test_api.py`	11	Full journey: register → plan → generate (202 + poll) → feedback → approve → analytics, on a live TestClient
`test_monthly_regeneration.py`	6	Regenerate-all uses the configured provider and the saved brief; failures mark the request FAILED and keep the old batch
`test_webhook.py`	6	Scheduler webhook authentication, idempotency, concurrent-tick rejection
`test_demo_accounts.py`	6	Seeded personas, idempotent seeding, history and cycle integrity
`test_scheduler.py`	4	Reminder days, idempotent cycle preparation, catch-up after missed runs, plan auto-extension
`test_health_commit.py`	3	Deployed commit reported from `RENDER_GIT_COMMIT`: set, unset (null, never guessed) and fallback/whitespace handling
Total	**166**	**All passing**

Gate	Result
Backend tests	**166/166 pass**
TypeScript typecheck	**0 errors**
ESLint (`next lint`)	**0 warnings, 0 errors**
Production build	**Builds and deploys on Vercel**
CI pipeline	**GitHub Actions: pytest on Python 3.10 + 3.11, frontend typecheck + build on Node 20**
Deployed API	**Health green; provider `groq` , model `openai/gpt-oss-120b` ; commit reported**
Browser end-to-end	**Login → dashboard → review → feedback on the deployed site and locally**

5. Bugs found and fixed during development

A deliberately test-driven build: these were caught by the golden tests, the behavioural suites or live verification, and fixed rather than skipped.

Issue	Resolution
SQLAlchemy enum column receiving a Python enum object	Pass the portable string value through the `SAEnum` converter
FastAPI asserted on `-> None` response annotations	Removed the annotations (204 responses)
Naive/aware datetime mixing from SQLite	Added a `UTCDateTime` TypeDecorator, applied to 20 columns
ICS generation overflowed the hour on all-day events	Replaced `start + timedelta(hours=1)` with a bounded end time
Generated candidates were never embedded → similarity always 0	Embed each candidate in `_make_candidate`
Ambiguous truth value on a NumPy blob comparison	Explicit `is not None` guard (regression caught by tests)
`data_dir` resolved to `backend/data` instead of the repo root	Anchored to the repository root in both config and templates
Cycle horizon off-by-one when filling 12 cycles	Loop to `CYCLE_HORIZON+1`, break when full
MMR could return two candidates from the same style arm	Added a `groups` parameter preferring unrepresented arms
Weight sanitiser did not converge to bounds	Iterative clip + renormalise loop
Frontend: TypeScript and unused-import lint errors	Fixed; typecheck and build now clean
Post-MVP issues found under real use	
Monthly regeneration silently substituted template text for model output, then presented it as an LLM batch	Removed the silent fallback entirely: template only when `MM_ALLOW_TEMPLATE_FALLBACK` is on, failures marked FAILED with the reason stored, and the review header labels template batches
Review page could not show which brief or model produced a batch	Persisted the brief at request time, exposed provider/model/source/timestamp, and rendered the "Monthly Post Details" card from the stored brief
Cycle-date calculation compared an explicit start month against the real clock	Anchor the reference date to the caller's explicit start; added a clock-independent regression test
`/review` flashed "Nothing waiting for review" before redirecting	Hold the skeleton while the resolution is in flight instead of rendering the empty state
`/review` followed a cached cycle pointer to a batch that no longer existed (404)	Validate the request before redirecting; skip stale pointers and fall through to the empty state
Review screen showed "nothing to review" despite an awaiting cycle	A stale pre-restart uvicorn process was serving old code; restarted with current code
Silent token refresh failed after 15 minutes → 401 on feedback	API base used `127.0.0.1` while the page is `localhost` (different site, so the `SameSite=Lax` refresh cookie was never sent). Switched to `localhost`; cookie became `None+Secure` under `MM_ENVIRONMENT=production` for cross-domain hosting

No way to tell whether a deployment had actually landed	`/admin/health` now reports the running commit from the host's environment variable, with tests for the set and unset cases

6. The AI engine, concretely

Candidates are generated by a **hosted language model** — in production Groq serving `openai/gpt-oss-120b`, with `openai/gpt-oss-20b` and a Qwen model as automatic fallbacks if the account cannot call the primary. Every provider sits behind one adapter interface (Groq, Gemini, OpenAI, Anthropic, Ollama), each with its own circuit breaker, retry schedule (1 s, 3 s, 9 s with jitter), tolerant JSON parsing and cost metering.

Concern	Behaviour
Provider chain	Primary provider → any other configured hosted provider → (only if opted in) the template engine. A provider whose breaker is open is skipped, never retried in a loop.
Failure policy	No silent substitution. If every provider fails, the request is marked `FAILED`, the reason is stored on the request, the previous batch survives, and the UI says the regeneration failed.
Template fallback	64 templates across 5 slot banks, used only with `MM_ALLOW_TEMPLATE_FALLBACK=true`. Template batches are labelled as such on the review screen — never presented as model output.
Provenance	Each request stores provider, model, source (`schedule`, `regenerate`), creation time and any fallback reason.
Cost control	One call per batch (six candidates), token metering and a monthly budget guardrail; no per-candidate calls.
Determinism	Ranking is seeded (Monte-Carlo seed 42, Thompson RNG, tone training) so identical inputs and weights produce identical rankings even when the text itself varies.

> **Why this matters for the report:** earlier revisions described the template engine as the default provider. That is no longer true. The template engine is now an explicit, opt-in safety net; production runs on a hosted model, and the review UI always states which produced the batch.

7. Documented deviations from the blueprint

Blueprint recommendation	Shipped instead	Reason
PostgreSQL + pgvector	SQLite default, Postgres-ready via `MM_DATABASE_URL`	Zero-setup development and free-tier hosting; schema and tests identical either way. Managed Postgres is the next step for durability.
sentence-transformers embeddings	MiniLM in development; deterministic 384-d hashing embedder in the deployed build	Instant startup and no model download on a small hosting instance; same dimensionality, so every formula and golden test is unchanged
spaCy + YAKE preprocessing	Own `text.py` (spaCy as an optional extra)	Heavy native dependencies; the needed operations are small and testable in isolation
shadcn/ui components	Own component library on Tailwind tokens	Fewer moving parts, exact control of the design tokens, no CLI-generated drift
Hosted LLM in the hot path	**Adopted** — Groq in production, behind the adapter chain	The blueprint was right: model-written copy is materially better. The template engine remains as an opt-in fallback rather than the default.

8. Repository & deliverables

- **GitHub:** `https://github.com/vxrsha1501/monthly-muse` — branch `main`, 137 tracked files.
- **Live:** frontend `https://monthly-muse-bay.vercel.app`; API `https://monthlymuse-api.onrender.com` (Render web service + scheduler worker).
- **Documentation:** README (quickstart + configuration matrix), `docs/api.md`, the Technology Stack reference, the Presentation Script, and this report.
- **DevOps:** `backend/Dockerfile`, `frontend/Dockerfile`, `docker-compose.yml` (Postgres + API + worker + frontend), `.github/workflows/ci.yml`.
- **Seed data:** 51 occasions, 20 topics, 64 templates, an 800-row tone training set, and four demo personas (`demo@`, `pro@`, `studio@`, `org@monthlymuse.app`) with two published months of history.
- **Notebooks:** cosine calibration, tone classifier, ablation study, simulated users.

9. Deployment status

Piece	Platform	State
Frontend	Vercel	**Deployed, auto-deploys on every push to `main`**
API	Render web service (Docker)	**Deployed, health green, provider `groq`; deploys are currently manual — auto-deploy still needs the repo connected through the Render GitHub App**
Worker	Render service	**Runs the scheduler loop; the authenticated webhook tick is available when a host suspends background schedulers**
Database	SQLite file on the service	**Fine for a demo; free-tier restarts can wipe it. Managed Postgres is the durability fix**
Secrets	Platform environment variables	**Provider keys outside the repository; nothing committed**

> **Known constraints:** free-tier web services sleep after ~15 minutes of inactivity, which is why the worker runs in a restart loop; scheduler preparation is idempotent so a sleeping and waking worker cannot duplicate a cycle or a reminder. Because the Render service is not connected as a GitHub App repository, backend deploys are triggered manually — the health endpoint's `commit` field makes the deployed revision verifiable.

10. Remaining work

Item	Priority
Move production data to managed Postgres so a restart cannot wipe history	High — the only real durability gap
Connect the Render service to GitHub and set Auto-Deploy to "On Commit" (API and worker)	Medium — removes manual deploy steps
Add a Playwright end-to-end test covering login → review → select → analytics in CI	Medium — the API journey is already covered
Provider-degradation alerting (fallback rate, breaker trips) surfaced in analytics	Medium — provenance data already exists
Polish offline template wording (minor grammatical artefacts)	Low — affects the fallback only
Version 2 items (§15.2): platform publisher, engagement import, teams, dark mode, autopilot	Out of MVP scope

MonthlyMuse · Project Progress Report · 166 tests · 58 route handlers · 16 tables · 12 routes · 137 files · deployed November 2026